Trade-offs & What's Next

What I cut and why

Several capabilities are conspicuously absent, and none of them were forgotten.

Personalization ML was the most significant cut. A learned model for weighting metrics by customer behavior would be more defensible than my rule-based growth/scale profiles, but it requires click-stream or engagement data that doesn't exist yet. The two profiles are a reasonable prior, not a substitute—they encode assumptions about what growth-stage versus scale-stage brands care about, and those assumptions need to be validated before training anything on top of them.

GA4 funnel metrics (sessions, bounce rate, conversion steps) weren't in the dataset. I could have synthesized them, but synthetic data in a prototype creates false confidence in the architecture. The ranker is designed to accept additional metric columns; the plumbing is ready when the data is.

Email delivery, A/B testing infrastructure, and real-time alerting were all out of scope by deliberate choice rather than time pressure. Delivery infrastructure is a solved problem and adds no signal in a prototype. A/B testing report variants requires real engagement data to measure against—running it now would generate noise, not insight. Real-time alerting adds operational complexity that isn't justified until we know daily batch isn't sufficient; for a morning digest use case, it probably is.

Hourly granularity and SKU-level breakdowns are dataset constraints, not design constraints. The architecture doesn't preclude them.

What I'd build next

Three architectural upgrades, each replacing a specific structural weakness of v1 with a known-better, industry-validated alternative. Then two product-layer items contingent on customer-feedback data.

1. **STL or Prophet decomposition for the baseline.** The current rolling-mean-with-DoW-adjustment is correct for the prototype scope, but it can't model annual seasonality (Diwali, EOSS, Christmas), can't adapt fast to genuine level shifts, and isn't robust when bad days enter the baseline window. STL separates trend, seasonality, and residual properly; Prophet adds explicit holiday calendars and changepoint detection. Z-score the residual, not the raw value. Used in production by Netflix, Uber, LinkedIn, DataDog Watchdog, and Booking.com. The reason I didn't ship it in v1: STL needs two cycles of the longest seasonality, and annual seasonality needs two years of history. The current dataset is 160 days.
2. **CausalImpact for the attribution question.** Right now the system hedges causation because the bivariate correlation is 0.12. The grown-up answer is Bayesian Structural Time Series (Brodersen et al. 2015, Google Research) — given an intervention date and a control series, it estimates the counterfactual with credible intervals. Instead of "revenue is up but we can't infer", the system says "Meta budget +50% on Dec 4 drove +12% over counterfactual, 95% CI 7–17%."

Used internally at Google, published in a Walmart Global Tech case study, standard at Lifesight, Measured, Recast, and every other incrementality vendor. Didn't ship because the dataset has no labelled interventions and no clean control series.

3. **Cross-metric story grouping.** Today five correlated findings on the same date become five bullets. Meta CPM up + CPC up + CTR down + ROAS down is one story (auction pressure), not four findings. Add a clustering step between ranker and LLM — pairwise similarity on (source, channel, campaign, direction), connected components, then a small LLM call to synthesize a story headline. Anodot, Outlier.ai (acquired by Salesforce), OutOfTheBlue, and DataDog Watchdog all do this. Marginal value at the prototype's 5–10 findings per day; large win at multi-tenant scale.

4. **Engagement feedback loop.** Thumbs up/down on each finding, plus click tracking on any linked metric. None of the per-customer-learning items below this point are worth building without it.

5. **Metric-preference learning from engagement signals.** Once finding-level feedback exists, replace the global business-weight constants with per-customer learned weights from a simple logistic regression over "finding type × customer segment × engaged/ignored." Removes the cold-start assumption from the personalisation layer.

Full v1-to-v2 evolution including implementation sketches, trade-off analysis, my research path to each upgrade, and a comparison table of what production systems at Netflix, Booking, Uber, LinkedIn, DataDog, Anodot, OutOfTheBlue, Google, and Walmart actually use is in `OPTIMIZATION_JOURNEY.md`.

What I need to learn from real customers first

Before any of the above gets prioritized, I have questions that prototypes can't answer:

- **Do they read daily or skip to weekly?** The entire batch cadence assumption depends on this. If customers open Monday's report on Wednesday, daily freshness is wasted infrastructure cost.
- **Which finding types drive follow-up actions versus get ignored?** I have hypotheses about which z-score thresholds feel actionable. I don't have evidence.
- **What's the tolerance for false positives versus false negatives?** A brand that checks their dashboard obsessively wants high recall and will tolerate noise. A brand that only looks when the report flags something needs high precision. These are different products wearing the same interface.
- **Do growth-stage and scale-stage brands actually behave differently in response to findings?** My profile split is a reasonable prior. It may be wrong. If scale-stage brands respond strongly to acquisition findings too, the segmentation logic needs revisiting before it gets encoded anywhere more permanent.

Where this breaks at scale

I want to be specific rather than vague about this.

At 100 customers, two problems emerge. Sequential LLM calls at roughly 3 seconds each means a two-report-per-customer batch runs approximately 10 minutes. That's manageable but not comfortable, and it's fully blocking. Parallelization is straightforward but requires async refactoring and rate limit management. The subtler problem at 100 customers is report differentiation—with rule-based profiles and a shared prompt structure, reports across similar customers will start to look nearly identical. Customers will notice, and it will correctly feel like a mail merge rather than an analyst.

At 10,000 customers, cost becomes the binding constraint. At roughly $0.15 per report and two reports per customer, the daily LLM spend is approximately $3,000. Prompt caching on the static system prompt and metric context structure is the first mitigation, but it requires careful prompt architecture to maximize cache hit rate. Beyond cost, nightly batch SLA risk becomes real—a slow tenant cohort or a model latency spike could push delivery past the morning window customers expect. Tenant data isolation also needs to be explicit in the architecture; right now it's implicit in how I've structured the notebook. At 10,000 tenants, "implicit" is a liability.

None of these are surprising failure modes, which is why I'm stating them plainly. The prototype is honest about what it is: a working proof of concept for the ranking and generation logic, not a production system.